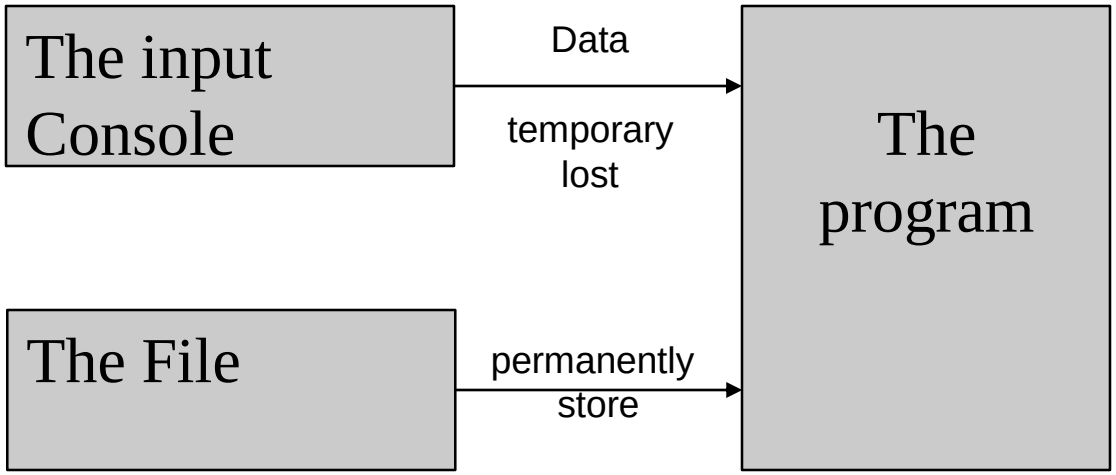
Chapter 14 Text IO

Objectives

- To discover file/directory properties, to delete and rename files/directories, and to create directories using the **File** class (§ 14.10).
- To write data to a file using the **PrintWriter** class (§ 14.11.1).
- To read data from a file using the **Scanner** class (§ 14.11.3).
- To understand how data is read using a **Scanner** (§ 14.11.4).
- To develop a program that replaces text in a file (§ 14.11.5).

You need to enter 100 students' names and IDs, How would you do that?



The Files

```
import java.io.*;
import java.util.*;
public class Example {
    public static void main(String[] args) throws IOException {

        File file = new File("newFile.txt");
        file.createNewFile();

        PrintWriter write = new PrintWriter(file);

        write.println("Hello CPCS203 Students");
        write.println("Welcome to Java File I/O");
        write.close();
}}
```

The File Class

- The File class is intended to provide an abstraction that deals with most of the machine-dependent complexities of files and path names in a machine-independent fashion.
- The filename is a String. The File class is a wrapper class (use primitive data types as objects) for the file name and its directory path.
- The File Class:
  - Contains methods for obtaining file and directory properties and for renaming and deleting files and directories.
  - Does not contain methods for reading and writing the file contents.

Create a File Object

- +File(pathname: String) \\ the whole path
  - File file = **new** File(**"c:\\book\\test.dat"**);
- +File(pathname: String) \\ Directory
  - File file = new File("c:\\book");
- +File(parent: String, child: String)
  - File file = new File("c:\\book", "test.dat");
- +File(parent: File, child: String)

```
File parent = new File("c:\\book");
File file   = new File(parent, "test.dat");
```

Obtaining file properties and manipulating file

- +exists(): boolean

⇒ True if file or directory represented by File object exists.

- +canRead(): boolean

⇒ True if file represented by File object exists and can be read.

- +canWrite(): boolean

⇒ True if file represented by File object exists and can be written.

Obtaining file properties and manipulating file

- +isDirectory(): boolean

⇒ True if File object represents a directory.

- +isFile(): boolean

⇒ True if File object represents a file.

- +isAbsolute(): boolean

⇒ True if File object is created using an absolute path name.

- +isHidden(): boolean

⇒ True if file represented in File object is hidden.

Obtaining file properties and manipulating file

- +lastModified(): long

⇒ The time that the file was last modified.

- +length(): long

⇒ The size of the file (0 if it does not exist or if it's a directory).

- +listFile(): File[]

⇒ the files under the directory.

- +delete(): boolean

⇒ True if the deletion succeeds.

Obtaining file properties and manipulating file

- +renameTo(dest: File): boolean

⇒ Renames the file or directory to the specified name. Returns true if the operation succeeds.

```
file.renameTo(new File(”NewName”));
```

- +getName(): String

⇒ Returns the last name of the complete directory and file name represented by the File object. For example, new File("c:\\book\\test.dat").getName() returns test.dat.

- +getParent(): String

⇒ Returns the complete parent directory of the current directory or the file represented by the File object. For example, new File("c:\\book\\test.dat").getParent() returns c:\book.

File Paths

- Every file is placed in a directory in the file system.
- A path is a way to get to a particular file or directory in a file system and it can be:
  - Relative path (does not start with the root element, relative to the current location)
  - Absolute path (full path)
  - Canonical path
- The path meanings:
  - / is the root of the current drive;
  - ./ is the current directory;
  - ../ is the parent of the current directory

Absolute Path

- An absolute file name (or full name) contains a file name with its complete path and drive letter (from root of the file system).
- Absolute path may contain meta characters like . and .. to represent current and parent directory.
- Absolute file names are machine dependent.
- Example (Windows OS)
  - c:\book\Welcome.java (absolute file for Welcome.Java) backslash
  - c:\book (directory path)
- Example (UNIX OS)
  - /home/liang/book/Welcome.java (absolute file for Welcome.Java)
  - /home/liang/book (directory path) forward slash

Canonical path

- All canonical paths are absolute (but not all absolute paths are canonical).
- A single file existing on a system can have many different paths that refer to it, but only one canonical path.
- Canonical gives a unique absolute path for a given file.

Obtaining file properties and manipulating file

+getPath(): String	Returns the complete directory and file name represented by the File object and it **may be relative to current directory.**
+getAbsolutePath(): String	Returns the complete **absolute** file or directory represented by the File object. A file or directory might have **several absolute paths.**
+getCanonicalPath(): String	Returns the same as getAbsolutePath() except that it removes **redundant names, such as "." and "..”**

getPath(): Example 1

We are given a file object of a file, we have to get the path of the file object.

```
File f  = new File("c:\\users\\program.txt");

        // Get the path of the given file f
        String path = f.getPath();

        // Display the file path of the file object
        System.out.println("File path : " + path);
```

getPath(): Example 1

We are given a file object of a file, we have to get the path of the file object.

```
File f  = new File("c:\\users\\program.txt");

        // Get the path of the given file f
        String path = f.getPath();

        // Display the file path of the file object
        System.out.println("File path : " + path);
```

Output:

File path : c:\users\program.txt

getPath(): Example 2

We are given a file object of a directory, we have to get the path of the file object.

```
// Create a file object
File f = new File("c:\\users\\program");

// Get the path of the given file f
String path = f.getPath();

// Display the file path of the file object
System.out.println("File path : " + path);
```

getPath(): Example 2

We are given a file object of a directory, we have to get the path of the file object.

```
// Create a file object
File f = new File("c:\\users\\program");

// Get the path of the given file f
String path = f.getPath();

// Display the file path of the file object
System.out.println("File path : " + path);
```

Output:

File path : c:\users\program

getAbsolutePath(): Example 1

A file named “program.txt” is in the present working directory.

```
// Create a file object
File f = new File("program.txt");

// Get the absolute path of file f
String absolute = f.getAbsolutePath();

// Display the file path of the file object
// and also the file path of the absolute file
System.out.println("Original path: " + f.getPath());
System.out.println("Absolute path: " + absolute);
```

getAbsolutePath(): Example 1

A file named "program.txt" is in the present working directory.

```
// Create a file object
File f = new File("program.txt");

// Get the absolute path of file f
String absolute = f.getAbsolutePath();

// Display the file path of the file object
// and also the file path of the absolute file
System.out.println("Original path: " + f.getPath());
System.out.println("Absolute path: " + absolute);
```

Output:

```
Original Path: program.txt
Absolute Path: C:\Users\pc\eclipse-workspace1\arnab\program.txt
```

getAbsolutePath(): Example 2

A directory named “program” is in the present working directory

```
// Create a file object
File f = new File("program");

// Get the absolute path of file f
String absolute = f.getAbsolutePath();

// Display the file path of the file object
// and also the file path of absolute file
System.out.println("Original path: "
                                        + f.getPath());
System.out.println("Absolute path: "
                                        + absolute);
```

getAbsolutePath(): Example 2

A directory named “program” is in the present working directory

```
// Create a file object
File f = new File("program");

// Get the absolute path of file f
String absolute = f.getAbsolutePath();

// Display the file path of the file object
// and also the file path of absolute file
System.out.println("Original path: "
                   + f.getPath());

System.out.println("Absolute path: "
                   + absolute);
```

Output:

```
Original Path: program
Absolute Path: C:\Users\pc\eclipse-workspace1\arnab\program
```

getAbsolutePath(): Example 3

A file named “f:\program.txt” in the “f:\” directory

```
// Create a file object
File f = new File("f:\\program.txt");

// get the absolute path of file f
String absolute = f.getAbsolutePath();

// display the file path of the file object
// and also the file path of absolute file
System.out.println("Original path: "
                                        + f.getPath());
System.out.println("Absolute path: "
                                        + absolute);
```

getAbsolutePath(): Example 3

A file named “f:\program.txt” in the “f:\” directory

```
// Create a file object
File f = new File("f:\\program.txt");

// get the absolute path of file f
String absolute = f.getAbsolutePath();

// display the file path of the file object
// and also the file path of absolute file
System.out.println("Original path: "
                   + f.getPath());

System.out.println("Absolute path: "
                   + absolute);
```

Output:

Original file path: f:\program.txt
Absolute file path: f:\program.txt

getCanonicalPath(): Example 1

We have a File object with a specified path we will try to find its canonical path

```
// Create a file object
File f = new File("c:\\program");

// Get the Canonical path of file f
String canonical = f.getCanonicalPath();

// Display the file path of the file object
// and also the file path of Canonical file
System.out.println("Original file path : "
                                        + f.getPath());
System.out.println("Canonical file path : "
                                        + canonical);
```

getCanonicalPath(): Example 1

We have a File object with a specified path we will try to find its canonical path

```
// Create a file object
File f = new File("c:\\program");

// Get the Canonical path of file f
String canonical = f.getCanonicalPath();

// Display the file path of the file object
// and also the file path of Canonical file
System.out.println("Original file path : "
                                        + f.getPath());
System.out.println("Canonical file path : "
                                        + canonical);
```

Output

```
Original file path : c:\program
Canonical file path : c:\program
```

getCanonicalPath(): Example 2

We have a File object with a specified path we will try to find its canonical path .

```
// Create a file object
File f = new File("c:\\users\\..\\program");

// Get the Canonical path of file f
String canonical = f.getCanonicalPath();

// Display the file path of the file object
// and also the file path of Canonical file
System.out.println("Original file path : "
                                        + f.getPath());
System.out.println("Canonical file path : "
                                        + canonical);
```

getCanonicalPath(): Example 2

We have a File object with a specified path we will try to find its canonical path .

```
// Create a file object
File f = new File("c:\\users\\..\\program");

// Get the Canonical path of file f
String canonical = f.getCanonicalPath();

// Display the file path of the file object
// and also the file path of Canonical file
System.out.println("Original file path : "
                                        + f.getPath());
System.out.println("Canonical file path : "
                                        + canonical);
```

Output

```
Original file path : c:\users\..\program
Canonical file path : c:\program
```

Example

If **dir1** and **dir2** are two directories and **test.txt** file is in **dir1**.
C:\dir1 and C:\dir2

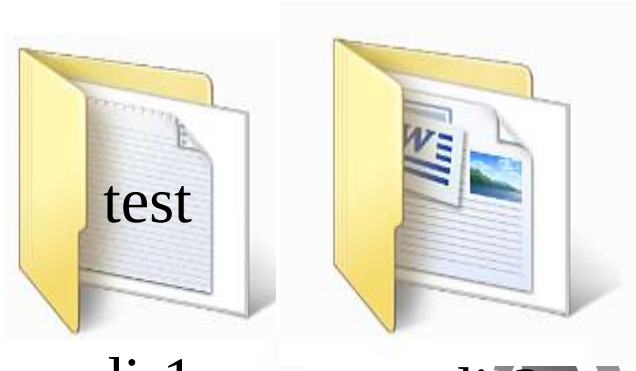
Possible absolute paths that refer to test.txt are:

C:\dir1\test.txt

C:\dir1\.\test.txt

C:\dir2\..\dir1\test.txt

(note: this means C go to dir2 then go to the parent of dir2 (C) then go to dir1)



Only **one canonical path refer to test.txt:**

C:\dir1\test.txt

What are the possible paths?

Problem: Explore File Properties

```
public class TestFileClass {
  public static void main(String[] args) {
    java.io.File file = new java.io.File("image/us.gif");
    System.out.println("Does it exist? " + file.exists());
    System.out.println("The file has " + file.length() + " bytes");
    System.out.println("Can it be read? " + file.canRead());
    System.out.println("Can it be written? " + file.canWrite());
    System.out.println("Is it a directory? " + file.isDirectory());
    System.out.println("Is it a file? " + file.isFile());
    System.out.println("Is it absolute? " + file.isAbsolute());
    System.out.println("Is it hidden? " + file.isHidden());
    System.out.println("Absolute path is " +
      file.getAbsolutePath());
    System.out.println("Last modified on " +
      new java.util.Date(file.lastModified()));
  }
}
```

Does it exist? True

The file has 543 bytes

Can it be read? False

Can it be written? False

Is it a directory? False

Is it a file? True

Is it absolute? False

Is it hidden? False

Absolute path is c:\ags10e\etext2014\firsttime\image\us.gif

Last modified on Wed Dec 31 19:00:00 EST 1969

TestFileClass

Run

Test Your Knowledge

```
File f1 = new File(".././test.txt");
System.out.println("Path : " + f1.getPath());
System.out.println("Absolute path : " + f1.getAbsolutePath());
System.out.println("Canonical path : " + f1.getCanonicalPath());
System.out.println("Get Name  : " + f1.getName());
System.out.println("Get parent : " + f1.getParent());
```

```
Path : .././test.txt
Absolute path : /Users/areejalhothali/NetBeansProjects/JavaApplication17/.././test.txt
Canonical path : /Users/areejalhothali/NetBeansProjects/test.txt
Get Name  : test.txt
Get parent : ../.
```

Question

- What is wrong about creating a File object using the following statement?

```
new File("c:\book\test.dat");
```

- The \ is a special character. It should be written as \\ in Java using the Escape sequence.

Can you use the **File** class for I/O???

Text I/O

A File object encapsulates the properties of a file or a path, but does not contain the methods for reading/writing data from/to a file. In order to perform I/O, you need to create objects using appropriate Java I/O classes. The objects contain the methods for reading/writing data from/to a file. This section introduces how to read/write strings and numeric values from/to a text file using the:

- Scanner class
- PrintWriter class

Writing Data Using PrintWriter

- **+PrintWriter(file: File) ➔** creates a PrintWriter object for the specified file.
- **+PrintWriter(filename: String) ➔** creates a PrintWriter object for the specified file name string.
- **+print(s: String): void ➔** Writes a string to the file.
- **+print(c: char): void ➔** Writes a character to the file.
- **+print(cArray: char[]): void ➔** Writes an array of characters to the file.
- **+print(i: int): void ➔** Writes an int value to the file.
- **+print(l: long): void ➔** Writes a long value to the file.
- **+print(f: float): void ➔** Writes a float value to the file.
- **+print(d: double): void ➔** Writes a double value to the file.
- **+print(b: boolean): void ➔** Writes a boolean value to the file.
- **Also contains the overloaded println methods ➔** like print but additionally prints a line separator.
- **Also contains the overloaded printf methods (format).**

Writing Data Using PrintWriter

```
public class WriteData {
  public static void main(String[] args) throws Exception {
    java.io.File file = new java.io.File("scores.txt");
    if (file.exists()) {
      System.out.println("File already exists");
      System.exit(0);
    }

    // Create a file
    java.io.PrintWriter output = new java.io.PrintWriter(file);

    // Write formatted output to the file
    output.print("John T Smith ");
    output.println(90);
    output.print("Eric K Jones ");
    output.println(85);

    // Close the file
    output.close();
  }
}
```

Check if file exist

https://liveexample.pearsoncmg.com/codeanimation/WriteData.html?

Writing Data Using PrintWriter

The program starts the execution from the main method.

```
1  public class WriteData {
2    public static void main(String[] args) throws java.io.IOException {
3      java.io.File file = new java.io.File("scores.txt");
4      if (file.exists()) {
5        System.out.println("File already exists");
6        System.exit(0);
7      }
8
9      // Create a file
10     java.io.PrintWriter output = new java.io.PrintWriter(file);
11
12     // Write formatted output to the file
13     output.print("John T Smith ");
14     output.println(90);
15     output.print("Eric K Jones ");
16     output.println(85);
17
18     // Close the file
19     output.close();
20   }
21 }
```

Writing Data Using PrintWriter

```
1  public class WriteData {
2    public static void main(String[] args) throws java.io.IOException {
3      java.io.File file = new java.io.File("scores.txt");
4      if (file.exists()) {
5        System.out.println("File already exists");
6        System.exit(0);
7      }
8
9      // Create a file
10     java.io.PrintWriter output = new java.io.PrintWriter(file);
11
12     // Write formatted output to the file
13     output.print("John T Smith ");
14     output.println(90);
15     output.print("Eric K Jones ");
16     output.println(85);
17
18     // Close the file
19     output.close();
20   }
21 }
```

The statement creates a File object named file for "scores.txt."

Writing Data Using PrintWriter

```
1  public class WriteData {
2    public static void main(String[] args) throws java.io.IOException {
3      java.io.File file = new java.io.File("scores.txt");
4      if (file.exists()) {
5        System.out.println("File already exists");
6        System.exit(0);
7      }
8
9      // Create a file
10     java.io.PrintWriter output = new java.io.PrintWriter(file);
11
12     // Write formatted output to the file
13     output.print("John T Smith ");
14     output.println(90);
15     output.print("Eric K Jones ");
16     output.println(85);
17
18     // Close the file
19     output.close();
20   }
21 }
```

The statement tests if the file exists. Assume the file does not exist.

If the file exists, the program will exit.

Writing Data Using PrintWriter

```
1  public class WriteData {
2    public static void main(String[] args) throws java.io.IOException {
3      java.io.File file = new java.io.File("scores.txt");
4      if (file.exists()) {
5        System.out.println("File already exists");
6        System.exit(0);
7      }
8
9      // Create a file
10     java.io.PrintWriter output = new java.io.PrintWriter(file);
11
12     // Write formatted output to the file
13     output.print("John T Smith ");
14     output.println(90);
15     output.print("Eric K Jones ");
16     output.println(85);
17
18     // Close the file
19     output.close();
20   }
21 }
```

The statement tests if the file exists. Assume the file does not exist.

If the file exists, this statement destroy the existi
So be extra careful when creating a PrintWriter

Writing Data Using PrintWriter

```
1  public class WriteData {
2    public static void main(String[] args) throws java.io.IOException {
3      java.io.File file = new java.io.File("scores.txt");
4      if (file.exists()) {
5        System.out.println("File already exists");
6        System.exit(0);
7      }
8
9      // Create a file
10     java.io.PrintWriter output = new java.io.PrintWriter(file);
11
12     // Write formatted output to the file
13     output.print("John T Smith ");
14     output.println(90);
15     output.print("Eric K Jones ");
16     output.println(85);
17
18     // Close the file
19     output.close();
20   }
21 }
```

String "John T Smith " is written to the file.

File scores.txt

John T Smith 90
Eric K Jones 85

Writing Data Using PrintWriter

```
1  public class WriteData {
2    public static void main(String[] args) throws java.io.IOException {
3      java.io.File file = new java.io.File("scores.txt");
4      if (file.exists()) {
5        System.out.println("File already exists");
6        System.exit(0);
7      }
8
9      // Create a file
10     java.io.PrintWriter output = new java.io.PrintWriter(file);
11
12     // Write formatted output to the file
13     output.print("John T Smith ");
14     output.println(90);
15     output.print("Eric K Jones ");
16     output.println(85);
17
18     // Close the file
19     output.close();
20   }
21 }
```

String "John T Smith " is written to the file.

File scores.txt

John T Smith 90
Eric K Jones 85

Writing Data Using PrintWriter

```
1  public class WriteData {
2    public static void main(String[] args) throws java.io.IOException {
3      java.io.File file = new java.io.File("scores.txt");
4      if (file.exists()) {
5        System.out.println("File already exists");
6        System.exit(0);
7      }
8
9      // Create a file
10     java.io.PrintWriter output = new java.io.PrintWriter(file);
11
12     // Write formatted output to the file
13     output.print("John T Smith ");
14     output.println(90);
15     output.print("Eric K Jones ");
16     output.println(85);
17
18     // Close the file
19     output.close();
20   }
21 }
```

The statement creates a File obje named file for "scores.txt."

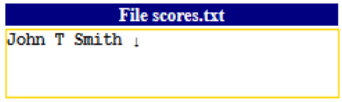
Writing Data Using PrintWriter

```
1  public class WriteData {
2    public static void main(String[] args) throws java.io.IOException {
3      java.io.File file = new java.io.File("scores.txt");
4      if (file.exists()) {The statement tests if the file exists. Assume the file does not exist.
5        System.out.println("File already exists");
6        System.exit(0);
7      }
8
9      // Create a file
10     java.io.PrintWriter output = new java.io.PrintWriter(file);
11
12     // Write formatted output to the file
13     output.print("John T Smith ");
14     output.println(90);
15     output.print("Eric K Jones ");
16     output.println(85);
17
18     // Close the file
19     output.close();
20   }
21 }
```

Writing Data Using PrintWriter

```
1  public class WriteData {
2    public static void main(String[] args) throws java.io.IOException {
3      java.io.File file = new java.io.File("scores.txt");
4      if (file.exists()) {
5        System.out.println("File already exists");
6        System.exit(0);
7      }
8
9      // Create a file
10     java.io.PrintWriter output = new java.io.PrintWriter(file);
11
12     // Write formatted output to the file
13     output.print("John T Smith ");
14     output.println(90);
15     output.print("Eric K Jones ");
16     output.println(85);
17
18     // Close the file
19     output.close();
20   }
21 }
```

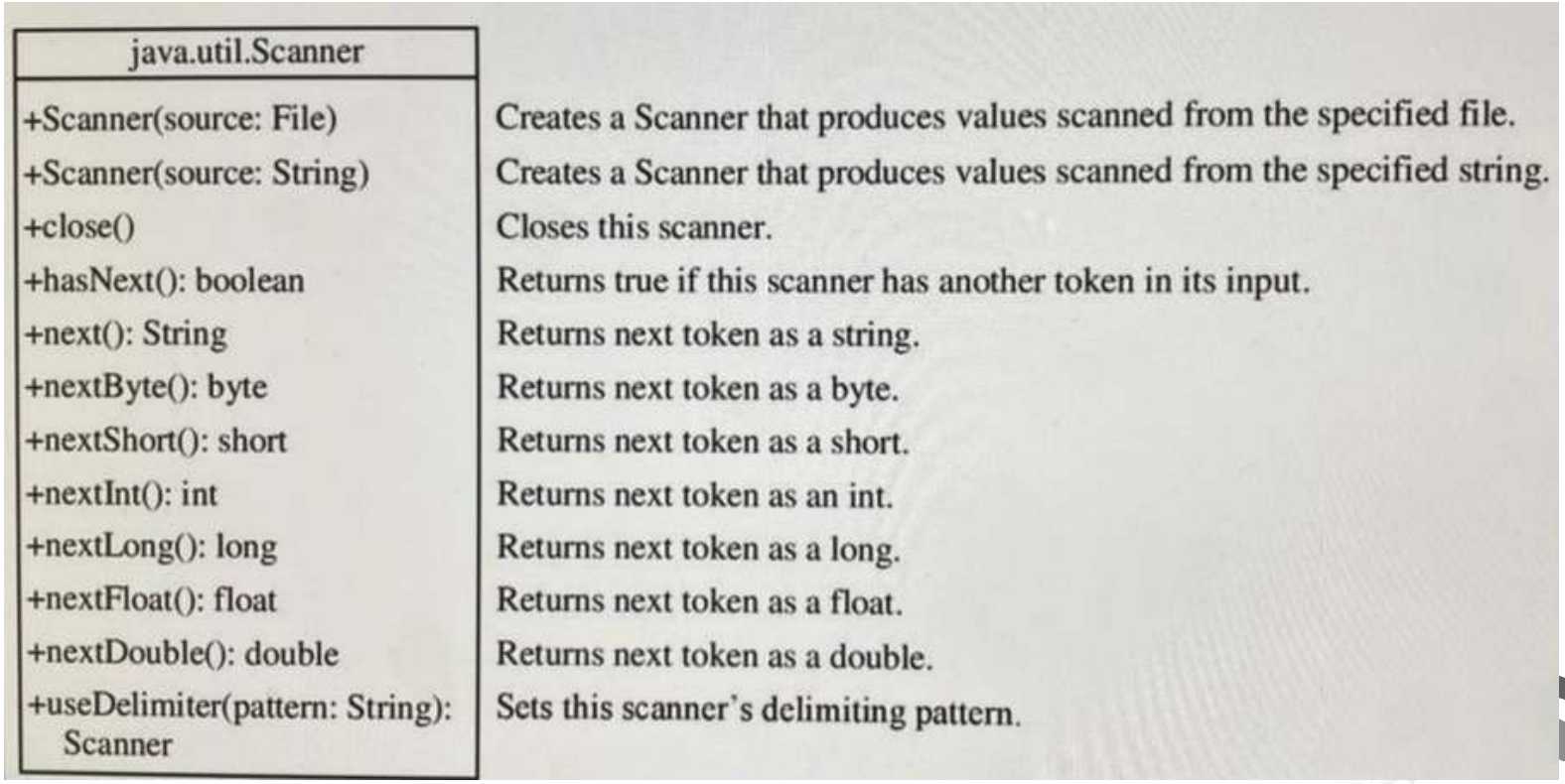
String "John T Smith " is written to the file.



Don't forget to close the output to ensure data is saved in the file.

Check the book for Closing Resources Automatically using try-with-resources

Reading Data Using Scanner



java.util.Scanner	
+Scanner(source: File)	Creates a Scanner that produces values scanned from the specified file.
+Scanner(source: String)	Creates a Scanner that produces values scanned from the specified string.
+close()	Closes this scanner.
+hasNext(): boolean	Returns true if this scanner has another token in its input.
+next(): String	Returns next token as a string.
+nextByte(): byte	Returns next token as a byte.
+nextShort(): short	Returns next token as a short.
+nextInt(): int	Returns next token as an int.
+nextLong(): long	Returns next token as a long.
+nextFloat(): float	Returns next token as a float.
+nextDouble(): double	Returns next token as a double.
+useDelimiter(pattern: String): Scanner	Sets this scanner's delimiting pattern.

Token-reading methods

+nextLine: String **Returns a line ending with the line separator from this scanner.**

Note:

next( ) reads a string separated by delimiters.

nextLine( ) reads a line ending with a line separator.

By default, the delimiters are whitespace characters.

Reading Data Using Scanner

- **+hasNextInt(): boolean**
- **+hasNextBoolean(): boolean**
- **+hasNextDouble(): boolean**
- **+hasNextFloat(): Boolean**
- **+hasNextFloat(): Boolean**

→ **Return true if and only if this scanner's next token is a valid (int, boolean, double, float)**

- **+hasNext(): Boolean (Doesn't check the type)**

```
 1  import java.util.Scanner;
 2
 3  public class ReadData {
 4    public static void main(String[] args) throws Exception {
 5      // Create a File instance
 6      java.io.File file = new java.io.File("scores.txt");
 7
 8      // Create a Scanner for the file
 9      Scanner input = new Scanner(file);
10
11      // Read data from a file
12      while (input.hasNext()) {
13        String firstName = input.next();
14        String mi = input.next();
15        String lastName = input.next();
16        int score = input.nextInt();
17        System.out.println(
18          firstName + " " + mi + " " + lastName + " " + score);
19      }
20
21      // Close the file
22      input.close();
23    }
24  }
```

The statement creates a File object named file for "scores.txt."

```
1  import java.util.Scanner;
2
3  public class ReadData {
4    public static void main(String[] args) throws Exception {
5      // Create a File instance
6      java.io.File file = new java.io.File("scores.txt");
7
8      // Create a Scanner for the file
9      Scanner input = new Scanner(file);
10
11     // Read data from a file
12     while (input.hasNext()) {
13       String firstName = input.next();
14       String mi = input.next();
15       String lastName = input.next();
16       int score = input.nextInt();
17       System.out.println(
18         firstName + " " + mi + " " + lastName + " " + score);
19     }
20
21     // Close the file
22     input.close();
23   }
24 }
```

The statement creates a File object named file for "scores.txt."

No Scanner object will be created and a FileNotFoundException will be thrown.

```java
import java.util.Scanner;

public class ReadData {
  public static void main(String[] args) throws Exception {
    // Create a File instance
    java.io.File file = new java.io.File("scores.txt");

    // Create a Scanner for the file
    Scanner input = new Scanner(file);

    // Read data from a file
    while (input.hasNext()) {
      String firstName = input.next();
      String mi = input.next();
      String lastName = input.next();
      int score = input.nextInt();
      System.out.println(
        firstName + " " + mi + " " + lastName + " " + score);
    }

    // Close the file
    input.close();
  }
}
```

Assume the file exists. The statement creates a Scanner object for reading data from the file. The red colored character in the file indicates the file pointer where a new read starts.

File scores.txt

```
John T Smith 90
Eric K Jones 85
```

```
 1  import java.util.Scanner;
 2
 3  public class ReadData {
 4    public static void main(String[] args) throws Exception {
 5      // Create a File instance
 6      java.io.File file = new java.io.File("scores.txt");
 7
 8      // Create a Scanner for the file
 9      Scanner input = new Scanner(file);
10
11      // Read data from a file
12      while (input.hasNext()) {
13        String firstName = input.next();
14        String mi = input.next();
15        String lastName = input.next();
16        int score = input.nextInt();
17        System.out.println(
18          firstName + " " + mi + " " + lastName + " " + score);
19      }
20
21      // Close the file
22      input.close();
23    }
24  }
```

The statement reads John from the file to firstName.
The file pointer is moved to T.

File scores.txt

```
John T Smith 90
Eric K Jones 85
```

```
1  import java.util.Scanner;
2
3  public class ReadData {
4    public static void main(String[] args) throws Exception {
5      // Create a File instance
6      java.io.File file = new java.io.File("scores.txt");
7
8      // Create a Scanner for the file
9      Scanner input = new Scanner(file);
10
11     // Read data from a file
12     while (input.hasNext()) {
13       String firstName = input.next();
14       String mi = input.next();
15       String lastName = input.next();
16       int score = input.nextInt();
17       System.out.println(
18         firstName + " " + mi + " " + lastName + " " + score);
19     }
20
21     // Close the file
22     input.close();
23   }
24 }
```

The statement reads John from the file to firstName.
The file pointer is moved to T.

File scores.txt

```
John T Smith 90
Eric K Jones 85
```

```
1  import java.util.Scanner;
2
3  public class ReadData {
4    public static void main(String[] args) throws Exception {
5      // Create a File instance
6      java.io.File file = new java.io.File("scores.txt");
7
8      // Create a Scanner for the file
9      Scanner input = new Scanner(file);
10
11     // Read data from a file
12     while (input.hasNext()) {
13       String firstName = input.next();
14       String mi = input.next();
15       String lastName = input.next();
16       int score = input.nextInt();
17       System.out.println(
18         firstName + " " + mi + " " + lastName + " " + score);
19     }
20
21     // Close the file
22     input.close();
23   }
24 }
```

The statement reads John from the file to firstName.
The file pointer is moved to T.

File scores.txt

```
John T Smith 90
Eric K Jones 85
```

```
1  import java.util.Scanner;
2
3  public class ReadData {
4    public static void main(String[] args) throws Exception {
5      // Create a File instance
6      java.io.File file = new java.io.File("scores.txt");
7
8      // Create a Scanner for the file
9      Scanner input = new Scanner(file);
10
11     // Read data from a file
12     while (input.hasNext()) {
13       String firstName = input.next();
14       String mi = input.next();
15       String lastName = input.next();
16       int score = input.nextInt();
17       System.out.println(
18         firstName + " " + mi + " " + lastName + " " + score);
19     }
20
21     // Close the file
22     input.close();
23   }
24 }
```

The statement reads John from the file to firstName.
The file pointer is moved to T.

File scores.txt

```
John T Smith 90
Eric K Jones 85
```

```java
import java.util.Scanner;

public class ReadData {
  public static void main(String[] args) throws Exception {
    // Create a File instance
    java.io.File file = new java.io.File("scores.txt");

    // Create a Scanner for the file
    Scanner input = new Scanner(file);

    // Read data from a file
    while (input.hasNext()) {
      String firstName = input.next();
      String mi = input.next();
      String lastName = input.next();
      int score = input.nextInt();
      System.out.println(
        firstName + " " + mi + " " + lastName + " " + score);
    }

    // Close the file
    input.close();
  }
}
```

The statement reads John from the file to firstName.
The file pointer is moved to T.

File scores.txt

```
John T Smith 90
Eric K Jones 85
```

```java
import java.util.Scanner;

public class ReadData {
  public static void main(String[] args) throws Exception {
    // Create a File instance
    java.io.File file = new java.io.File("scores.txt");

    // Create a Scanner for the file
    Scanner input = new Scanner(file);

    // Read data from a file
    while (input.hasNext()) {
      String firstName = input.next();
      String mi = input.next();
      String lastName = input.next();
      int score = input.nextInt();
      System.out.println(
        firstName + " " + mi + " " + lastName + " " + score);
    }

    // Close the file
    input.close();
  }
}
```

The statement reads John from the file to firstName. The file pointer is moved to T.

File scores.txt

```
John T Smith 90
Eric K Jones 85
```

```
1  import java.util.Scanner;
2
3  public class ReadData {
4    public static void main(String[] args) throws Exception {
5      // Create a File instance
6      java.io.File file = new java.io.File("scores.txt");
7
8      // Create a Scanner for the file
9      Scanner input = new Scanner(file);
10
11     // Read data from a file
12     while (input.hasNext()) {
13       String firstName = input.next();
14       String mi = input.next();
15       String lastName = input.next();
16       int score = input.nextInt();
17       System.out.println(
18         firstName + " " + mi + " " + lastName + " " + score);
19     }
20
21     // Close the file
22     input.close();
23   }
24 }
```

The statement reads John from the file to firstName.
The file pointer is moved to T.

File scores.txt

```
John T Smith 90
Eric K Jones 85
```

```
1  import java.util.Scanner;
2
3  public class ReadData {
4    public static void main(String[] args) throws Exception {
5      // Create a File instance
6      java.io.File file = new java.io.File("scores.txt");
7
8      // Create a Scanner for the file
9      Scanner input = new Scanner(file);
10
11     // Read data from a file
12     while (input.hasNext()) {
13       String firstName = input.next();
14       String mi = input.next();
15       String lastName = input.next();
16       int score = input.nextInt();
17       System.out.println(
18         firstName + " " + mi + " " + lastName + " " + score);
19     }
20
21     // Close the file
22     input.close();
23   }
24 }
```

The statement reads John from the file to firstName.
The file pointer is moved to T.

File scores.txt

```
John T Smith 90
Eric K Jones 85
```

```java
import java.util.Scanner;

public class ReadData {
  public static void main(String[] args) throws Exception {
    // Create a File instance
    java.io.File file = new java.io.File("scores.txt");

    // Create a Scanner for the file
    Scanner input = new Scanner(file);

    // Read data from a file
    while (input.hasNext()) {
      String firstName = input.next();
      String mi = input.next();
      String lastName = input.next();
      int score = input.nextInt();
      System.out.println(
        firstName + " " + mi + " " + lastName + " " + score);
    }

    // Close the file
    input.close();
  }
}
```

The statement reads John from the file to firstName.
The file pointer is moved to T.

File scores.txt

```
John T Smith 90
Eric K Jones 85
```

```
1  import java.util.Scanner;
2
3  public class ReadData {
4    public static void main(String[] args) throws Exception {
5      // Create a File instance
6      java.io.File file = new java.io.File("scores.txt");
7
8      // Create a Scanner for the file
9      Scanner input = new Scanner(file);
10
11     // Read data from a file
12     while (input.hasNext()) {
13       String firstName = input.next();
14       String mi = input.next();
15       String lastName = input.next();
16       int score = input.nextInt();
17       System.out.println(
18         firstName + " " + mi + " " + lastName + " " + score);
19     }
20
21     // Close the file
22     input.close();
23   }
24 }
```

The statement reads John from the file to firstName.
The file pointer is moved to T.

File scores.txt

```
John T Smith 90
Eric K Jones 85
```

```
 1  import java.util.Scanner;
 2
 3  public class ReadData {
 4    public static void main(String[] args) throws Exception {
 5      // Create a File instance
 6      java.io.File file = new java.io.File("scores.txt");
 7
 8      // Create a Scanner for the file
 9      Scanner input = new Scanner(file);
10
11      // Read data from a file
12      while (input.hasNext()) {
13        String firstName = input.next();
14        String mi = input.next();
15        String lastName = input.next();
16        int score = input.nextInt();
17        System.out.println(
18          firstName + " " + mi + " " + lastName + " " + score);
19      }
20
21      // Close the file
22      input.close();
23    }
24  }
```

The statement reads John from the file to firstName.
The file pointer is moved to T.

File scores.txt

```
John T Smith 90
Eric K Jones 85
```

```
1  import java.util.Scanner;
2
3  public class ReadData {
4    public static void main(String[] args) throws Exception {
5      // Create a File instance
6      java.io.File file = new java.io.File("scores.txt");
7
8      // Create a Scanner for the file
9      Scanner input = new Scanner(file);
10
11     // Read data from a file
12     while (input.hasNext()) {
13       String firstName = input.next();
14       String mi = input.next();
15       String lastName = input.next();
16       int score = input.nextInt();
17       System.out.println(
18         firstName + " " + mi + " " + lastName + " " + score);
19     }
20
21     // Close the file
22     input.close();
23   }
24 }
```

The statement reads John from the file to firstName.
The file pointer is moved to T.

File scores.txt

```
John T Smith 90
Eric K Jones 85
```

Test Your Knowledge

What is the output of the following code?

```
PrintWriter output = new PrintWriter(new File("input.txt"));
output.println(33.6);
output.println(33);
output.println(1 > 2);
output.println("Java");
output.close();

Scanner input=new Scanner(new File("input.txt"));
System.out.println(input.hasNextDouble()+" "+input.next());
System.out.println(input.hasNextInt());
System.out.println(input.hasNextBoolean() +" "+ input.next());
```

- true 33.6
- true
- false 33

Question

- Suppose you enter 45 57.8 789, then press the Enter key. Show the contents of the variables after the following code is executed.
- Scanner input = new Scanner(System.in);
- int intValue = input.nextInt();
- double doubleValue = input.nextDouble();
- String line = input.nextLine();

  - intValue contains 45.
  - doubleValue contains 57.8.
  - line contains ' ', '7', '8', '9'

Question

- Suppose you enter 45, press the Enter key, 57.8, press the Enter key, 789, and press the Enter key. Show the contents of the variables after the following code is executed.
- Scanner input = new Scanner(System.in);
- int intValue = input.nextInt();
- double doubleValue = input.nextDouble();
- String line = input.nextLine();

  - intValue contains 45.
  - doubleValue contains 57.8, and
  - line is empty

Question

If you have two files that have the following text what are the values of these two variables?

```
34  567
```

intValue ?

line ?

```
34
567
```

intValue ?

line ?

```
Scanner input= new Scanner(new File("test.txt"));
int intValue=input.nextInt();
String line= input.nextLine();
```

Answer

If you have two files that have the following text what are the values of these two variables?

```
34  567
```

intValue 34

line “ 567”

```
34
567
```

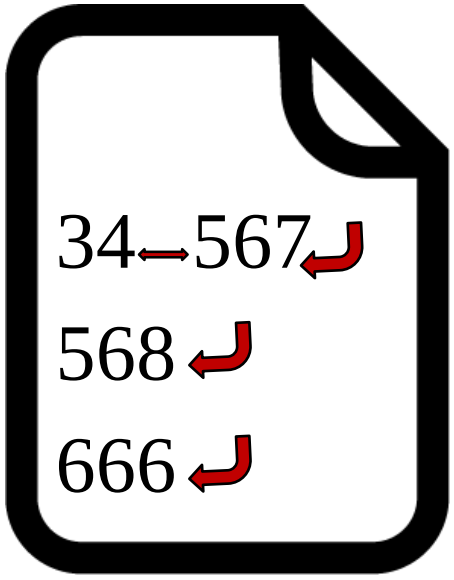
intValue 34

Line

```
Scanner input= new Scanner(new File("test.txt"));
int intValue=input.nextInt();
String line= input.nextLine();
```

Test Your Knowledge

A text file has the following text what is the output of the following code:



test.txt

```
File f1 = new File("test.txt");
Scanner input= new Scanner(f1);
System.out.println("intvalue1:"+input.next());
System.out.println("intvalue2:"+input.nextLine());
System.out.println("intvalue3:"+input.next());
System.out.println("intvalue4:"+input.nextLine());
```

run:
intvalue1:34
intvalue2: 567
intvalue3:568
intvalue4:

Problem: Replacing Text

Write a class named ReplaceText that replaces a string in a text file with a new string. The filename and strings are passed as an input as follows:

sourceFile.txt targetFile.txt oldString newString

For example, invoking

Source.txt Target.txt Hello Hi

replaces all the occurrences of oldString by newString in Source.txt and saves the new file in Target.txt .

```
package testing;
import java.io.*;
import java.util.*;
public class Testing {
  public static void main(String[] args) throws Exception {
        Scanner in = new Scanner(System.in);
        System.out.println("Please Enter the sourcefile , targetfile, oldstring, newstring" );
        String[] mainStrings = (in.nextLine()).split("\\s");
         if (mainStrings.length != 4) {     System.out.println( “You should enter sourceFile targetFile oldStr newStr");
                System.exit(1);    }
        // Check if source file exists
        File sourceFile = new File(mainStrings[0]);
         if (!sourceFile.exists()) {       System.out.println("Source file " + mainStrings[0] + " does not exist");
                System.exit(2);    }
          // Check if target file exists
   File targetFile = new File(mainStrings[1]);
   if (targetFile.exists()) {         System.out.println("Target file " + mainStrings[1] + " already exists");
          System.exit(3);    }
          // Create input and output files
    try (
        Scanner input = new Scanner(sourceFile);
       PrintWriter output = new PrintWriter(targetFile);
   ) {
     while (input.hasNext()) {
          String s1 = input.nextLine();
          String s2 = s1.replaceAll(mainStrings[2], mainStrings[3]);
          output.println(s2);
               }
               }
   }
}
```